Assignment 1 — Solutions

Christopher Elliott
A01526751

Rules used over and over.

$$\log(ab) = \log a + \log b, \qquad \log \frac{a}{b} = \log a - \log b, \qquad \log\left(a^k\right) = k \log a, \qquad \frac{n^a}{n^b} = n^{a-b}.$$

0.1

Throughout, log denotes logarithm in any fixed base greater than 1 (changing the base multiplies the log by a positive constant and does not affect these relations). Limits are as $n \to \infty$ through values large enough that every expression is positive.

Recall: $f = O(g)$ if f/g is bounded above, $f = \Omega(g)$ if f/g is bounded below by a positive constant, and $f = \Theta(g)$ if both hold. Equivalently, if $\lim f/g$ is a positive constant then $f = \Theta(g)$; if the limit is 0 then $f = O(g)$ only; if the limit is ∞ then $f = \Omega(g)$ only.

IDEA

For every pair, form the fraction f/g and simplify until you can see where it heads as n grows. Then read the answer off the fraction test: 0 means O only, ∞ means Ω only, a positive constant means Θ.

(a) $f(n) = n - 100$, $g(n) = n - 200$.

$$\frac{f(n)}{g(n)} = \frac{n - 100}{n - 200} \to 1.$$

So $f = \Theta(g)$.

(b) $f(n) = n^{1/2}$, $g(n) = n^{2/3}$.

$$\frac{f(n)}{g(n)} = n^{1/2 - 2/3} = n^{-1/6} \to 0.$$

So $f = O(g)$ only.

(c) $f(n) = 100n + \log n$, $g(n) = n + (\log n)^2$.

$$\frac{f(n)}{g(n)} = \frac{100 + (\log n)/n}{1 + (\log n)^2/n} \to 100.$$

So $f = \Theta(g)$.

(d) $f(n) = n \log n$, $g(n) = 10n \log(10n)$.

$$\frac{f(n)}{g(n)} = \frac{\log n}{10(\log 10 + \log n)} \to \frac{1}{10}.$$

So $f = \Theta(g)$.

IDEA

The steps in (d): cancel n, split $\log(10n) = \log 10 + \log n$, then divide the top and bottom by $\log n$ to get

$$\frac{1}{10\left(\frac{\log 10}{\log n} + 1\right)}.$$

$\log 10$ is fixed and $\log n$ keeps growing, so $\frac{\log 10}{\log n} \to 0$, leaving $\frac{1}{10(0+1)} = \frac{1}{10}$. Parts (e) and (f) use the same log rules.

(e) $f(n) = \log(2n)$, $g(n) = \log(3n)$.

$$\frac{f(n)}{g(n)} = \frac{\log 2 + \log n}{\log 3 + \log n} \to 1.$$

So $f = \Theta(g)$.

(f) $f(n) = 10 \log n$, $g(n) = \log(n^2)$.

$$\frac{f(n)}{g(n)} = \frac{10 \log n}{2 \log n} = 5.$$

So $f = \Theta(g)$.

(g) $f(n) = n^{1.01}$, $g(n) = n \log^2 n$.

$$\frac{f(n)}{g(n)} = \frac{n^{0.01}}{\log^2 n} \to \infty,$$

since any positive power of n grows faster than any polylogarithm. So $f = \Omega(g)$ only.

IDEA

Parts (g), (h), (i) and (k) all end the same way: after cancelling, a positive power of n sits over a power of $\log n$. Any positive power of n, even $n^{0.01}$, eventually beats any power of $\log n$, so the fraction heads to ∞.

(h) $f(n) = n^2 / \log n$, $g(n) = n(\log n)^2$.

$$\frac{f(n)}{g(n)} = \frac{n}{(\log n)^3} \to \infty.$$

So $f = \Omega(g)$ only.

(i) $f(n) = n^{0.1}$, $g(n) = (\log n)^{10}$.

$$\frac{f(n)}{g(n)} = \frac{n^{0.1}}{(\log n)^{10}} \to \infty.$$

So $f = \Omega(g)$ only.

(j) $f(n) = (\log n)^{\log n}$, $g(n) = n/\log n$.

Taking the base-2 logarithm,

$$\log_2 f(n) = \log n \cdot \log_2(\log n), \qquad \log_2 g(n) = \log_2 n - \log_2(\log n).$$

The first tends to infinity faster than the second ($\log n \cdot \log_2(\log n) = \omega(\log n)$), so $f/g \to \infty$. Thus $f = \Omega(g)$ only.

IDEA

In (j) and (p) the fraction is too tangled to simplify, so compare logs instead: $\log(f/g) = \log f - \log g$. If that difference heads to $+\infty$ then so does f/g; if it heads to $-\infty$ then $f/g \to 0$. Writing $L = \log_2 n$, part (j) has $f = L^L$ and $g = 2^L/L$, and

$$\log_2 f - \log_2 g = L \log_2 L - (L - \log_2 L) = L(\log_2 L - 1) + \log_2 L \to \infty.$$

(k) $f(n) = \sqrt{n}$, $g(n) = (\log n)^3$.

$$\frac{f(n)}{g(n)} = \frac{n^{1/2}}{(\log n)^3} \to \infty.$$

So $f = \Omega(g)$ only.

(l) $f(n) = n^{1/2}$, $g(n) = 5^{\log_2 n}$.

$$5^{\log_2 n} = (2^{\log_2 5})^{\log_2 n} = n^{\log_2 5},$$

and $\log_2 5 \approx 2.32 > 1/2$, so

$$\frac{f(n)}{g(n)} = n^{1/2 - \log_2 5} \to 0.$$

So $f = O(g)$ only.

IDEA

The identity behind (l) is $a^{\log_b n} = n^{\log_b a}$. It holds because $a = b^{\log_b a}$, so $a^{\log_b n} = b^{\log_b a \cdot \log_b n} = \left(b^{\log_b n}\right)^{\log_b a} = n^{\log_b a}$. It turns an awkward exponential into a plain power of n, after which you subtract exponents.

(m) $f(n) = n \cdot 2^n$, $g(n) = 3^n$.

$$\frac{f(n)}{g(n)} = n\left(\frac{2}{3}\right)^n \to 0,$$

because exponential decay dominates any linear factor. So $f = O(g)$ only.

(n) $f(n) = 2^n$, $g(n) = 2^{n+1}$.

$$\frac{f(n)}{g(n)} = \frac{2^n}{2 \cdot 2^n} = \frac{1}{2}.$$

So $f = \Theta(g)$.

(o) $f(n) = n!$, $g(n) = 2^n$.

For $n \geq 4$,

$$\frac{n!}{2^n} = \frac{1}{2} \cdot \frac{2}{2} \cdot \frac{3}{2} \cdots \frac{n}{2} \geq \frac{n}{4} \cdot \left(\frac{3}{2}\right)^{n-3} \to \infty,$$

since each factor from 3 onward is at least $3/2$ and the last factor is $n/2$. So $f = \Omega(g)$ only.

IDEA

In (o), both $n!$ and 2^n are products of n numbers, so pair them up factor by factor. From $\frac{3}{2}$ onward every factor is bigger than 1 and they keep growing, so the product runs off to ∞.

(p) $f(n) = (\log n)^{\log n}$, $g(n) = 2^{(\log_2 n)^2}$.

$$\log_2 f(n) = \log n \cdot \log_2(\log n) = O(\log n \cdot \log \log n), \qquad \log_2 g(n) = (\log_2 n)^2.$$

Since $\log n \cdot \log \log n = o((\log n)^2)$, we have $\log_2 f = o(\log_2 g)$, hence $f/g \to 0$. So $f = O(g)$ only.

(q) $f(n) = \sum_{i=1}^{n} i^k$, $g(n) = n^{k+1}$, for a fixed constant $k \geq 0$.

Upper bound. The sum has n terms, and each term satisfies $i^k \leq n^k$ because $i \leq n$ and $k \geq 0$. Hence

$$f(n) = \sum_{i=1}^{n} i^k \leq n \cdot n^k = n^{k+1}.$$

Lower bound. Keep only the terms with $\lceil n/2 \rceil \leq i \leq n$; the discarded terms are positive, so this can only decrease the sum. There are

$$n - \left\lceil \frac{n}{2} \right\rceil + 1 = \left\lfloor \frac{n}{2} \right\rfloor + 1 \geq \frac{n}{2}$$

such terms, and each satisfies $i^k \geq (n/2)^k$ because $i \geq n/2$ and $k \geq 0$. Hence

$$f(n) \geq \frac{n}{2} \cdot \left(\frac{n}{2}\right)^k = \left(\frac{n}{2}\right)^{k+1} = \frac{n^{k+1}}{2^{k+1}}.$$

Combining the two bounds and dividing by $g(n) = n^{k+1}$,

$$\frac{1}{2^{k+1}} \leq \frac{f(n)}{g(n)} \leq 1$$

for every $n \geq 1$. Both bounds are positive constants independent of n, so $f = \Theta(g)$.

IDEA

(q) is a trapping argument. Above: pretend every term is as big as the largest one, n^k. Below: throw away the smaller half of the terms and pretend each remaining one is only $(n/2)^k$. Both estimates are constant multiples of n^{k+1}. The ratio in fact heads to exactly $\frac{1}{k+1}$, which comparing the sum with the area under $y = x^k$ would show, but Θ does not need the exact constant.

0.2

IDEA

Each term is c times the one before, and one thing decides the size of the sum in each case. If the terms shrink ($c < 1$), the first term dominates. If they stay equal ($c = 1$), it is the number of terms. If they grow ($c > 1$), the last term dominates. The closed form comes from subtracting S from cS: the middle terms cancel and leave $cS - S = c^{n+1} - 1$.

Let $c > 0$ and

$$g(n) = \sum_{i=0}^{n} c^i = 1 + c + c^2 + \cdots + c^n.$$

The closed form is $g(n) = n + 1$ if $c = 1$, and

$$g(n) = \frac{c^{n+1} - 1}{c - 1}$$

if $c \neq 1$.

(a) Suppose $0 < c < 1$. Then $0 < c^{n+1} < 1$, so

$$1 \leq g(n) = \frac{1 - c^{n+1}}{1 - c} < \frac{1}{1 - c}.$$

The lower bound is the first term and the upper bound is the sum of the infinite series. Both bounds are positive constants independent of n, so $g(n) = \Theta(1)$.

(b) Suppose $c = 1$. Every term equals 1, and there are $n + 1$ of them, so

$$g(n) = n + 1.$$

Thus $n \leq g(n) \leq 2n$ for every $n \geq 1$, and $g(n) = \Theta(n)$.

(c) Suppose $c > 1$. The last term alone is a lower bound, and the closed form is an upper bound:

$$c^n \leq g(n) = \frac{c^{n+1} - 1}{c - 1} < \frac{c^{n+1}}{c - 1} = \frac{c}{c - 1}\, c^n.$$

The constant $c/(c - 1)$ is positive and independent of n, so $g(n) = \Theta(c^n)$.

0.3

IDEA

All three parts use induction. Because the Fibonacci rule looks back two steps, every proof needs two base cases, and the inductive step assumes the claim for both $n - 1$ and $n - 2$.

Let $\varphi = (1 + \sqrt{5})/2 \approx 1.618$. Then φ satisfies $\varphi^2 = \varphi + 1$, and $1 < \varphi < 2$.

(a) Claim: $F_n \geq 2^{n/2}$ for every $n \geq 6$.

Base cases:

- $F_6 = 8$ and $2^{6/2} = 2^3 = 8$, so $F_6 \geq 2^3$.
- $F_7 = 13$ and $2^{7/2} = 8\sqrt{2} \approx 11.31$, so $F_7 \geq 2^{7/2}$.

Inductive step: fix $n \geq 8$ and assume the claim holds for $n-1$ and $n-2$. Then

$$F_n = F_{n-1} + F_{n-2} \geq 2^{(n-1)/2} + 2^{(n-2)/2} = 2^{n/2}\big(2^{-1/2} + 2^{-1}\big).$$

Since $2^{-1/2} + 1/2 = 1/\sqrt{2} + 1/2 > 0.707 + 0.5 > 1$,

$$F_n \geq 2^{n/2}.$$

By induction the claim holds for all $n \geq 6$.

(b) Take $c = \log_2 \varphi$. Then $c < 1$ because $\varphi < 2$, and $2^{cn} = \varphi^n$.

Claim: $F_n \leq \varphi^n$ for every $n \geq 0$.

Base cases: $F_0 = 0 \leq 1 = \varphi^0$ and $F_1 = 1 \leq \varphi$.

Inductive step: if the claim holds for $n-1$ and $n-2$ with $n \geq 2$, then

$$F_n = F_{n-1} + F_{n-2} \leq \varphi^{n-1} + \varphi^{n-2} = \varphi^{n-2}(\varphi + 1) = \varphi^{n-2} \cdot \varphi^2 = \varphi^n,$$

using $\varphi + 1 = \varphi^2$. Thus $F_n \leq 2^{cn}$ for all $n \geq 0$.

IDEA

Why φ in (b): it satisfies $\varphi^2 = \varphi + 1$, which has the same shape as $F_n = F_{n-1} + F_{n-2}$, so the algebra in the inductive step closes exactly. Since $\varphi < 2$, the constant $c = \log_2 \varphi \approx 0.694$ is less than 1.

(c) The largest such constant is $c = \log_2 \varphi$.

First, this c works. Claim: $F_n \geq \varphi^{n-2}$ for every $n \geq 2$. The base cases are $F_2 = 1 = \varphi^0$ and $F_3 = 2 \geq \varphi$. If the claim holds for $n-1$ and $n-2$ with $n \geq 4$, then

$$F_n = F_{n-1} + F_{n-2} \geq \varphi^{n-3} + \varphi^{n-4} = \varphi^{n-4}(\varphi + 1) = \varphi^{n-2}.$$

Hence $F_n \geq \varphi^{-2} \cdot \varphi^n = \varphi^{-2} \cdot 2^{n \log_2 \varphi}$ for $n \geq 2$, so $F_n = \Omega(2^{cn})$.

It is the largest possible. Part (b) gives $F_n \leq \varphi^n$. If $c' > \log_2 \varphi$, then $2^{c'} > \varphi$ and

$$\frac{F_n}{2^{c'n}} \leq \left(\frac{\varphi}{2^{c'}}\right)^n \to 0,$$

so F_n is not $\Omega(2^{c'n})$.

IDEA

In (c), Ω allows a constant in front but not a bigger base. $F_n \geq \varphi^{n-2} = \varphi^n/\varphi^2 \approx 0.38\,\varphi^n$ is fine. But for any base $B > \varphi$, the ratio $(\varphi/B)^n$ shrinks to 0, just like $(2/3)^n$ in 0.1(m), so F_n cannot keep up with B^n.

1.1

IDEA

The worst case is three copies of the largest digit, $b-1$. Two digits can hold anything up to b^2-1. So the whole question is whether $3(b-1) \le b^2-1$, which rearranges to $(b-1)(b-2) \ge 0$.

In base $b \ge 2$, a single digit is an integer in $\{0, 1, \ldots, b-1\}$. The sum of any three of them is at most

$$3(b-1) = 3b-3.$$

The smallest three-digit number in base b is b^2 (written 100). It is enough to show $3b-3 < b^2$, or equivalently

$$b^2 - 3b + 2 = (b-1)(b-2) \ge 0,$$

which holds for every integer $b \ge 2$. Thus the sum is at most b^2-1, so it has at most two digits.

(For $b=2$ the largest sum is 3, written 11, which has exactly two digits.)

1.2

IDEA

The number of digits of N in base b is about $\log_b N$, and changing base multiplies a log by a constant: $\log_2 N = \log_2 10 \cdot \log_{10} N \approx 3.322 \log_{10} N$. So binary is about 3.322 times as long as decimal, which is under 4. The rest of the proof only makes sure rounding never pushes the ratio above 4.

For an integer $N \ge 1$, write $\ell_b(N)$ for the number of digits of N in base b. Then

$$b^{\ell_b(N)-1} \le N < b^{\ell_b(N)},$$

so $\ell_b(N) = \lfloor \log_b N \rfloor + 1$.

Let $d = \ell_{10}(N)$ and $b = \ell_2(N)$, and check $b \le 4d$.

If $d = 1$, then $1 \le N \le 9$ and the binary lengths are at most 4 (since $9 = 1001_2$), so $b \le 4d$.

If $d \ge 2$, then $10^{d-1} \le N < 10^d$, hence

$$b = \lfloor \log_2 N \rfloor + 1 \le \log_2 N + 1 < \log_2(10^d) + 1 = d \log_2 10 + 1.$$

Since $\log_2 10 < 3.33$ and $d \ge 2$,

$$d \log_2 10 + 1 < d \cdot 3.33 + \tfrac{d}{2} = 3.83\, d < 4d.$$

Thus $b < 4d$. As both are integers, $b \le 4d$.

For very large N the ratio of the lengths approaches the change-of-base constant:

$$\frac{\ell_2(N)}{\ell_{10}(N)} = \frac{\lfloor \log_2 N \rfloor + 1}{\lfloor \log_{10} N \rfloor + 1} \to \log_2 10 \approx 3.322.$$

1.3

IDEA

Count the most nodes a tree of depth D can hold: $1 + d + d^2 + \cdots + d^D$, the geometric sum from 0.2 with $c = d$. A short tree can only hold a few nodes, so having n nodes forces the depth up. Solving for D needs a log.

Assume $d \geq 2$. (If $d = 1$ the tree is a path, so its depth is exactly $n - 1$.) Take the depth of a node to be the number of edges from the root to that node, and the depth of the tree to be the maximum node depth.

In a d-ary tree of depth D, level k has at most d^k nodes. Therefore

$$n \leq \sum_{k=0}^{D} d^k = \frac{d^{D+1} - 1}{d - 1} < d^{D+1}.$$

Taking $\log_d$ of both sides of $n < d^{D+1}$ gives

$$D + 1 > \log_d n = \frac{\log n}{\log d},$$

so $D = \Omega(\log n / \log d)$.

The bound is achieved by filling levels from the root downward, so the minimum possible depth of a d-ary tree with n nodes is the smallest integer $D \geq 0$ such that $(d^{D+1} - 1)/(d - 1) \geq n$:

$$D = \left\lceil \log_d\big(1 + n(d-1)\big) \right\rceil - 1.$$

1.4

IDEA

Trap $n!$ between two simple powers. Above: replace every factor by n. Below: throw away the smaller half of the factors and replace each remaining one by $n/2$. Taking logs then turns both powers into multiples of $n \log n$.

Upper bound: each of the n factors in $n!$ is at most n, so $n! \leq n^n$ and

$$\log(n!) \leq n \log n = O(n \log n).$$

Lower bound: for $n \geq 2$, at least $\lfloor n/2 \rfloor$ factors of $n!$ are $\geq n/2$ (namely $\lceil n/2 \rceil, \ldots, n$). Hence

$$n! \geq \left(\frac{n}{2}\right)^{\lfloor n/2 \rfloor},$$

and

$$\log(n!) \geq \left\lfloor \frac{n}{2} \right\rfloor \log \frac{n}{2} = \left\lfloor \frac{n}{2} \right\rfloor (\log n - \log 2) = \Omega(n \log n).$$

Combining the two bounds, $\log(n!) = \Theta(n \log n)$.

1.5

IDEA

Do not compute the sum; trap it. Group the fractions into blocks between powers of two. Rounding every denominator in a block down to a power of two makes the block worth at most 1; rounding up makes it worth at least $\frac{1}{2}$. There are about $\log_2 n$ blocks, so both bounds are constant multiples of $\log n$.

Let $H_n = \sum_{i=1}^{n} 1/i$, and write $m = \lfloor \log_2 n \rfloor$, so $2^m \le n < 2^{m+1}$.

Upper bound. Decrease each denominator to the next lower power of two: the term 1 is 1, and for each $k \ge 0$ the terms with $2^k \le i < 2^{k+1}$ satisfy $1/i \le 1/2^k$. There are 2^k such indices, so that block sums to at most 1. Including the partial last block up to n,

$$H_n \le \sum_{k=0}^{m} 1 = m + 1 \le \log_2 n + 1 = O(\log n).$$

Lower bound. Increase each denominator to the next higher power of two: for each $k \ge 1$, the 2^{k-1} terms with $2^{k-1} < i \le 2^k$ satisfy $1/i \ge 1/2^k$, so that block sums to at least $1/2$. Since $n \ge 2^m$, the blocks for $k = 1, \ldots, m$ are entirely included, and

$$H_n \ge 1 + \sum_{k=1}^{m} \frac{1}{2} = 1 + \frac{m}{2} \ge 1 + \frac{\log_2 n - 1}{2} = \Omega(\log n).$$

Therefore $H_n = \Theta(\log n)$.

1.9

IDEA

Before doing anything else, rewrite $\equiv$ as an ordinary equation: $x \equiv x' \pmod N$ becomes $x = x' + aN$ for some integer a. After that both rules are plain algebra, and the goal each time is to show the difference is an integer times N.

By definition, $x \equiv y \pmod N$ means that N divides $x - y$, that is, $x - y = kN$ for some integer k.

Addition. Suppose $x \equiv x' \pmod N$ and $y \equiv y' \pmod N$. Then there are integers a and b such that

$$x - x' = aN, \qquad y - y' = bN.$$

Adding these identities,

$$(x + y) - (x' + y') = (a + b)N,$$

so N divides $(x + y) - (x' + y')$. Therefore $x + y \equiv x' + y' \pmod N$.

Multiplication. With the same integers a and b, write $x = x' + aN$ and $y = y' + bN$. Then

$$xy = (x' + aN)(y' + bN) = x'y' + N(x'b + y'a + abN),$$

so

$$xy - x'y' = N(x'b + y'a + abN).$$

Thus N divides $xy - x'y'$, and $xy \equiv x'y' \pmod N$.

1.10

IDEA

A multiple of N is automatically a multiple of anything that divides N.

Since $a \equiv b \pmod{N}$, there is an integer k such that $a - b = kN$. Since M divides N, there is an integer m such that $N = mM$. Substituting,

$$a - b = k(mM) = (km)M,$$

so M divides $a - b$. Therefore $a \equiv b \pmod{M}$.

References

[1] S. Dasgupta, C. H. Papadimitriou, and U. V. Vazirani. *Algorithms*. McGraw-Hill, 2008.

[2] Anthropic. *Claude* (model Claude Opus 5.5), accessed through Claude Code, October 2026. `https://claude.com/claude-code`

External resources and AI usage

The exercises are taken from Chapters 0 and 1 of [1].

I solved the exercises myself. I used Claude Opus 5.5, through Claude Code, to give me hints when I was stuck and to check my results [2].

The same tool was also used in preparing this document, for the following:

- converting the solutions and the exercise statements from Markdown to LaTeX;
- explaining the solutions and the concepts behind them (asymptotic notation, geometric sums, induction and modular arithmetic), including a step-by-step walkthrough and a set of practice questions;
- writing the list of rules that opens this document and the boxed Idea notes that introduce the solutions, and the further study notes kept in the study copy (a "Notation and methods" section and boxed Check and Watch out notes);
- changing the notation in the solution to 1.9 from x_0, y_0 to x', y' to match the exercise statement.